NovusOS

A Graphical UEFI Operating System for AArch64

Complete Technical Documentation

Language	Rust (nightly, #![no_std], #![no_main])
Architecture	AArch64 (ARM64) / ARMv8-A
Boot Protocol	UEFI Application (PE/COFF)
Display	UEFI GOP Framebuffer (1024x768x32bpp)
Input	UEFI SimpleTextInput Protocol
Target Platforms	QEMU (qemu-system-aarch64) + VMware Fusion
Author	Abhishek Bhatia
Date	May 24, 2026
Lines of Code	~1,500+ (Rust)

Table of Contents

1. Executive Summary

NovusOS is a graphical operating system written in Rust that runs as a UEFI application on AArch64 (ARM64) hardware. Unlike a traditional OS kernel that calls `ExitBootServices()` and takes full hardware control, NovusOS retains UEFI boot services throughout its execution, leveraging them for keyboard input, memory allocation, and system control. This is the same execution model used by the UEFI Shell itself.

NovusOS provides a full graphical desktop environment with a gradient background, status bar showing system information, and a bordered terminal window with an interactive command shell. It boots from a standard EFI System Partition, making it compatible with both QEMU and VMware Fusion on Apple Silicon.

Key Features

- UEFI GOP framebuffer with pixel-level drawing (fill_rect, put_pixel, scroll)
- Embedded 8x16 bitmap font with full ASCII rendering (128 glyphs)
- Graphical text console implementing core::fmt::Write for formatted output
- Keyboard input via UEFI SimpleTextInput protocol polling
- Interactive shell with line editing (type, backspace, enter)
- GUI desktop with vertical gradient, dark status bar, terminal window with shadow
- Built-in commands: help, ver, mem, uptime, cpuinfo, color, echo, clear, reboot, shutdown
- AArch64 architectural timer (CNTVCT_EL0) for uptime tracking
- UEFI memory map query showing total and available RAM
- CPU identification via MIDR_EL1 register decode
- Bootable image creation for QEMU and VMware Fusion

2. Architecture Overview

2.1 UEFI Application Model

NovusOS is compiled as a PE/COFF executable targeting `aarch64-unknown-uefi`. The UEFI firmware loads the binary from `EFI/BOOT/BOOTAA64.EFI` on the EFI System Partition, allocates memory, and calls the entry point. Boot services remain active throughout execution, providing:

- **Graphics Output Protocol (GOP)**: Framebuffer access for display rendering
- **SimpleTextInput Protocol**: Keyboard input without needing USB HID drivers
- **Memory Allocation**: Rust's alloc crate via uefi crate's global allocator
- **Timer**: boot::stall() for delays, CNTVCT_EL0 for uptime
- **Runtime Services**: System reset (reboot/shutdown) via runtime::reset()

2.2 Why Not ExitBootServices?

Calling `ExitBootServices()` transfers full hardware control to the OS but removes access to all UEFI protocols. This would require implementing: USB HID keyboard driver (XHCI host controller + HID class driver), interrupt controller driver (GICv3), page table management, and a heap allocator. By keeping boot services active, NovusOS avoids ~3,000 lines of driver code and can focus on the graphical environment.

2.3 Project Structure

Project Directory Layout

```
custom-os/
  Cargo.toml                    # Workspace root
  Makefile                      # Build + run targets
  rust-toolchain.toml           # Nightly + targets
  novusos/
    Cargo.toml                  # uefi crate dependency
    .cargo/config.toml          # aarch64-unknown-uefi target
    src/
      main.rs                   # UEFI entry, GOP init, event loop
      framebuffer.rs            # Pixel ops on GOP framebuffer
      font.rs                   # 8x16 bitmap font (128 glyphs)
      console.rs                # Graphical text console
      keyboard.rs               # UEFI keyboard polling
      desktop.rs                # Desktop gradient + status bar
      window.rs                 # Terminal window rendering
      shell.rs                  # Line editor + command dispatch
      commands.rs               # Built-in commands
      timer.rs                  # Uptime via CNTVCT_EL0
      memory.rs                 # UEFI memory map query
      color.rs                  # Color type + palette
  esp/efi/boot/
    BOOTAA64.EFI                # Compiled UEFI binary
  tools/
    mkiso.sh                    # Bootable image creation
```

```
generate_novusos_pdf.py      # This documentation generator
```

2.4 Build Infrastructure

NovusOS is built as a workspace member alongside the kernel. It uses a per-crate `.cargo/config.toml` to override the build target to `aarch64-unknown-uefi`, which produces a PE/COFF binary via Rust's built-in rust-lld linker.

novusos/Cargo.toml

```
[package]
name = "novusos"
version = "1.0.0"
edition = "2021"

[dependencies]
uefi = { version = "0.34", features = ["alloc", "global_allocator", "panic_handler"] }
log = "0.4"
```

novusos/.cargo/config.toml

```
[build]
target = "aarch64-unknown-uefi"
```

QEMU UEFI boot requires the EDK2 firmware image (`edk2-aarch64-code.fd`) loaded as a pflash drive, and the EFI binary placed in a FAT32 ESP directory that QEMU's VVFAT driver exposes.

Build Commands

```
# Build NovusOS
make build-novusos

# Run in QEMU with UEFI firmware
make run-novusos

# Create bootable disk image
make iso

# Run from disk image
make run-iso
```

3. Phase 1: UEFI Boot & GOP Framebuffer

The first phase establishes the UEFI entry point, locates the Graphics Output Protocol, selects the best available video mode (preferring 1024x768), and creates a Framebuffer abstraction for pixel-level drawing.

3.1 UEFI Entry Point

The `#[entry]` macro from the uefi crate generates the PE/COFF entry point. `uefi::helpers::init()` sets up logging, the global allocator, and the panic handler. The entry function returns `Status` to UEFI firmware.

3.2 GOP Initialization

GOP is located via `boot::get_handle_for_protocol()`. All available video modes are enumerated to find 1024x768, falling back to the highest resolution available. From the selected mode, we extract the framebuffer base pointer, dimensions, stride (pixels per scanline), and pixel format (BGR or RGB).

The pixel format is critical: UEFI's GOP can report BGR or RGB byte ordering. The Framebuffer driver adapts pixel writes based on `PixelFormat` to ensure correct colors regardless of firmware implementation.

3.3 Framebuffer Driver

The Framebuffer struct wraps the raw framebuffer pointer with safe pixel operations. Key detail: the pixel byte offset calculation uses `(y * stride + x) * 4`, where stride is in pixels (not bytes). All writes use `core::ptr::write_volatile` to prevent the compiler from eliding writes to memory-mapped I/O.

novusos/src/framebuffer.rs

```
use crate::color::Color;
use uefi::proto::console::gop::PixelFormat;

pub struct Framebuffer {
    base: *mut u8,
    pub width: usize,
    pub height: usize,
    stride: usize,
    pixel_format: PixelFormat,
}

unsafe impl Send for Framebuffer {}
unsafe impl Sync for Framebuffer {}

impl Framebuffer {
    pub fn new(
        base: *mut u8,
        width: usize,
        height: usize,
        stride: usize,
```

```
    pixel_format: PixelFormat,
) -&amp;gt; Self {
    Self { base, width, height, stride, pixel_format }
}

pub fn stride(&amp;amp;self) -&amp;gt; usize {
    self.stride
}

fn pixel_bytes(&amp;amp;self, color: Color) -&amp;gt; [u8; 4] {
    match self.pixel_format {
        PixelFormat::Bgr =&amp;gt; color.to_bgra(),
        PixelFormat::Rgb =&amp;gt; color.to_rgba(),
        _ =&amp;gt; color.to_bgra(),
    }
}

pub fn put_pixel(&amp;amp;mut self, x: usize, y: usize, color: Color) {
    if x &amp;gt;= self.width || y &amp;gt;= self.height {
        return;
    }
    let offset = (y * self.stride + x) * 4;
    let bytes = self.pixel_bytes(color);
    unsafe {
        let ptr = self.base.add(offset);
        core::ptr::write_volatile(ptr, bytes[0]);
        core::ptr::write_volatile(ptr.add(1), bytes[1]);
        core::ptr::write_volatile(ptr.add(2), bytes[2]);
        core::ptr::write_volatile(ptr.add(3), bytes[3]);
    }
}

pub fn fill_rect(&amp;amp;mut self, x: usize, y: usize, w: usize, h: usize, color: Color) {
    let bytes = self.pixel_bytes(color);
    let x_end = (x + w).min(self.width);
    let y_end = (y + h).min(self.height);

    for row in y..y_end {
        let row_offset = row * self.stride * 4;
        for col in x..x_end {
            let offset = row_offset + col * 4;
            unsafe {
                let ptr = self.base.add(offset);
                core::ptr::write_volatile(ptr, bytes[0]);
                core::ptr::write_volatile(ptr.add(1), bytes[1]);
                core::ptr::write_volatile(ptr.add(2), bytes[2]);
                core::ptr::write_volatile(ptr.add(3), bytes[3]);
            }
        }
    }
}

pub fn clear(&amp;amp;mut self, color: Color) {
    self.fill_rect(0, 0, self.width, self.height, color);
}

pub fn draw_hline(&amp;amp;mut self, x: usize, y: usize, w: usize, color: Color) {
    self.fill_rect(x, y, w, 1, color);
}

pub fn draw_vline(&amp;amp;mut self, x: usize, y: usize, h: usize, color: Color) {
    self.fill_rect(x, y, 1, h, color);
}

pub fn scroll_region_up(
```

```
        &amp;mut self,
        x: usize,
        y: usize,
        w: usize,
        h: usize,
        amount: usize,
        bg: Color,
    ) {
        if amount &gt;= h {
            self.fill_rect(x, y, w, h, bg);
            return;
        }

        for row in y..(y + h - amount) {
            let src_row = row + amount;
            let src_offset = (src_row * self.stride + x) * 4;
            let dst_offset = (row * self.stride + x) * 4;
            unsafe {
                core::ptr::copy(
                    self.base.add(src_offset),
                    self.base.add(dst_offset),
                    w * 4,
                );
            }
        }

        self.fill_rect(x, y + h - amount, w, amount, bg);
    }
}
```

3.4 Color System

Colors are represented as RGB triplets with conversion methods for both BGR and RGB pixel formats. A palette of named constants provides consistent theming.

novusos/src/color.rs

```
#[derive(Clone, Copy, Debug)]
pub struct Color {
    pub r: u8,
    pub g: u8,
    pub b: u8,
}

impl Color {
    pub const fn new(r: u8, g: u8, b: u8) -&gt; Self {
        Self { r, g, b }
    }

    pub fn to_bgra(self) -&gt; [u8; 4] {
        [self.b, self.g, self.r, 0xFF]
    }

    pub fn to_rgba(self) -&gt; [u8; 4] {
        [self.r, self.g, self.b, 0xFF]
    }
}

pub const BLACK: Color = Color::new(0, 0, 0);
pub const WHITE: Color = Color::new(255, 255, 255);
pub const DARK_BLUE: Color = Color::new(20, 30, 60);
pub const MEDIUM_BLUE: Color = Color::new(35, 50, 100);
pub const LIGHT_GRAY: Color = Color::new(200, 200, 200);
```

```
pub const DARK_GRAY: Color = Color::new(40, 40, 40);
pub const RED: Color = Color::new(220, 60, 60);
pub const GREEN: Color = Color::new(60, 200, 80);
pub const CYAN: Color = Color::new(80, 200, 220);
pub const YELLOW: Color = Color::new(220, 200, 60);
pub const TITLE_BAR: Color = Color::new(50, 50, 55);
pub const BORDER: Color = Color::new(100, 100, 110);
pub const CONTENT_BG: Color = Color::new(15, 15, 20);
pub const STATUS_BG: Color = Color::new(30, 30, 35);
pub const ACCENT: Color = Color::new(80, 140, 220);
```

4. Phase 2: Font Rendering & Text Console

4.1 Embedded 8x16 Bitmap Font

NovusOS embeds a classic 8x16 bitmap font as a compile-time constant array. Each of the 128 ASCII glyphs is represented by 16 bytes, where each byte encodes one row of 8 pixels. Bit 7 (MSB) is the leftmost pixel. The font data is based on the classic VGA/CP437 character set, providing clear rendering at the pixel level.

The `render_char()` function iterates over the 16 rows and 8 columns of each glyph, writing foreground or background color based on the corresponding bit. Characters outside the 0-127 ASCII range fall back to glyph 0 (null/blank).

novusos/src/font.rs

```
use crate::color::Color;
use crate::framebuffer::Framebuffer;

pub const GLYPH_WIDTH: usize = 8;
pub const GLYPH_HEIGHT: usize = 16;

pub fn render_char(fb: &amp;mut Framebuffer, x: usize, y: usize, ch: char, fg: Color, bg: Color) {
    let idx = ch as usize;
    let glyph = if idx &lt; 128 {
        &amp;FONT_DATA[idx * GLYPH_HEIGHT..(idx + 1) * GLYPH_HEIGHT]
    } else {
        &amp;FONT_DATA[0..GLYPH_HEIGHT]
    };

    for row in 0..GLYPH_HEIGHT {
        let bits = glyph[row];
        for col in 0..GLYPH_WIDTH {
            let color = if bits &amp; (0x80 &gt;&gt; col) != 0 { fg } else { bg };
            fb.put_pixel(x + col, y + row, color);
        }
    }
}

pub fn render_str(fb: &amp;mut Framebuffer, x: usize, y: usize, s: &amp;str, fg: Color, bg: Color) {
    let mut cx = x;
    for ch in s.chars() {
        if cx + GLYPH_WIDTH &gt; fb.width {
            break;
        }
        render_char(fb, cx, y, ch, fg, bg);
        cx += GLYPH_WIDTH;
    }
}

#[rustfmt::skip]
static FONT_DATA: [u8; 128 * GLYPH_HEIGHT] = [
    // 0x00 - NULL
    0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
    // 0x01
    0x00,0x00,0x7E,0x81,0xA5,0x81,0x81,0xBD,0x99,0x81,0x81,0x7E,0x00,0x00,0x00,0x00,
    // 0x02
```

```
0x00,0x00,0x7E,0xFF,0xDB,0xFF,0xFF,0xC3,0xE7,0xFF,0xFF,0x7E,0x00,0x00,0x00,0x00,
// 0x03
0x00,0x00,0x00,0x00,0x6C,0xFE,0xFE,0xFE,0xFE,0x7C,0x38,0x10,0x00,0x00,0x00,0x00,
// 0x04
0x00,0x00,0x00,0x00,0x10,0x38,0x7C,0xFE,0x7C,0x38,0x10,0x00,0x00,0x00,0x00,0x00,
// 0x05
0x00,0x00,0x00,0x18,0x3C,0x3C,0xE7,0xE7,0xE7,0x18,0x18,0x3C,0x00,0x00,0x00,0x00,
// 0x06
0x00,0x00,0x00,0x18,0x3C,0x7E,0xFF,0xFF,0x7E,0x18,0x18,0x3C,0x00,0x00,0x00,0x00,
// 0x07
0x00,0x00,0x00,0x00,0x00,0x00,0x18,0x3C,0x3C,0x18,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x08
0xFF,0xFF,0xFF,0xFF,0xFF,0xFF,0xE7,0xC3,0xC3,0xE7,0xFF,0xFF,0xFF,0xFF,0xFF,0xFF,
// 0x09
0x00,0x00,0x00,0x00,0x00,0x3C,0x66,0x42,0x42,0x66,0x3C,0x00,0x00,0x00,0x00,0x00,
// 0x0A
0xFF,0xFF,0xFF,0xFF,0xFF,0xC3,0x99,0xBD,0xBD,0x99,0xC3,0xFF,0xFF,0xFF,0xFF,0xFF,
// 0x0B
0x00,0x00,0x1E,0x0E,0x1A,0x32,0x78,0xCC,0xCC,0xCC,0xCC,0x78,0x00,0x00,0x00,0x00,
// 0x0C
0x00,0x00,0x3C,0x66,0x66,0x66,0x66,0x3C,0x18,0x7E,0x18,0x18,0x00,0x00,0x00,0x00,
// 0x0D
0x00,0x00,0x3F,0x33,0x3F,0x30,0x30,0x30,0x30,0x70,0xF0,0xE0,0x00,0x00,0x00,0x00,
// 0x0E
0x00,0x00,0x7F,0x63,0x7F,0x63,0x63,0x63,0x63,0x67,0xE7,0xE6,0xC0,0x00,0x00,0x00,
// 0x0F
0x00,0x00,0x00,0x18,0x18,0xDB,0x3C,0xE7,0x3C,0xDB,0x18,0x18,0x00,0x00,0x00,0x00,
// 0x10
0x00,0x80,0xC0,0xE0,0xF0,0xF8,0xFE,0xF8,0xF0,0xE0,0xC0,0x80,0x00,0x00,0x00,0x00,
// 0x11
0x00,0x02,0x06,0x0E,0x1E,0x3E,0xFE,0x3E,0x1E,0x0E,0x06,0x02,0x00,0x00,0x00,0x00,
// 0x12
0x00,0x00,0x18,0x3C,0x7E,0x18,0x18,0x18,0x7E,0x3C,0x18,0x00,0x00,0x00,0x00,0x00,
// 0x13
0x00,0x00,0x66,0x66,0x66,0x66,0x66,0x66,0x66,0x00,0x66,0x66,0x00,0x00,0x00,0x00,
// 0x14
0x00,0x00,0x7F,0xDB,0xDB,0xDB,0x7B,0x1B,0x1B,0x1B,0x1B,0x1B,0x00,0x00,0x00,0x00,
// 0x15
0x00,0x7C,0xC6,0x60,0x38,0x6C,0xC6,0xC6,0x6C,0x38,0x0C,0xC6,0x7C,0x00,0x00,0x00,
// 0x16
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xFE,0xFE,0xFE,0xFE,0x00,0x00,0x00,0x00,
// 0x17
0x00,0x00,0x18,0x3C,0x7E,0x18,0x18,0x18,0x7E,0x3C,0x18,0x7E,0x00,0x00,0x00,0x00,
// 0x18
0x00,0x00,0x18,0x3C,0x7E,0x18,0x18,0x18,0x18,0x18,0x18,0x18,0x00,0x00,0x00,0x00,
// 0x19
0x00,0x00,0x18,0x18,0x18,0x18,0x18,0x18,0x18,0x7E,0x3C,0x18,0x00,0x00,0x00,0x00,
// 0x1A
0x00,0x00,0x00,0x00,0x00,0x18,0x0C,0xFE,0x0C,0x18,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x1B
0x00,0x00,0x00,0x00,0x00,0x30,0x60,0xFE,0x60,0x30,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x1C
0x00,0x00,0x00,0x00,0x00,0x00,0xC0,0xC0,0xC0,0xFE,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x1D
0x00,0x00,0x00,0x00,0x00,0x28,0x6C,0xFE,0x6C,0x28,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x1E
0x00,0x00,0x00,0x00,0x10,0x38,0x38,0x7C,0x7C,0xFE,0xFE,0x00,0x00,0x00,0x00,0x00,
// 0x1F
0x00,0x00,0x00,0x00,0xFE,0xFE,0x7C,0x7C,0x38,0x38,0x10,0x00,0x00,0x00,0x00,0x00,
// 0x20 - SPACE
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x21 - !
0x00,0x00,0x18,0x3C,0x3C,0x3C,0x18,0x18,0x18,0x00,0x18,0x18,0x00,0x00,0x00,0x00,
// 0x22 - "
0x00,0x63,0x63,0x63,0x22,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
```

```
// 0x23 - #
0x00,0x00,0x00,0x36,0x36,0x7F,0x36,0x36,0x7F,0x36,0x36,0x00,0x00,0x00,0x00,0x00,
// 0x24 - $
0x0C,0x0C,0x3E,0x63,0x61,0x60,0x3E,0x03,0x03,0x43,0x63,0x3E,0x0C,0x0C,0x00,0x00,
// 0x25 - %
0x00,0x00,0x00,0x00,0x43,0x63,0x06,0x0C,0x18,0x30,0x63,0x61,0x00,0x00,0x00,0x00,
// 0x26 - &amp;
0x00,0x00,0x1C,0x36,0x36,0x1C,0x6E,0x3B,0x33,0x33,0x33,0x6E,0x00,0x00,0x00,0x00,
// 0x27 - '
0x00,0x0C,0x0C,0x0C,0x06,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x28 - (
0x00,0x00,0x0C,0x18,0x30,0x30,0x30,0x30,0x30,0x30,0x18,0x0C,0x00,0x00,0x00,0x00,
// 0x29 - )
0x00,0x00,0x30,0x18,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x18,0x30,0x00,0x00,0x00,0x00,
// 0x2A - *
0x00,0x00,0x00,0x00,0x00,0x66,0x3C,0xFF,0x3C,0x66,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x2B - +
0x00,0x00,0x00,0x00,0x00,0x18,0x18,0x7E,0x18,0x18,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x2C - ,
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x18,0x18,0x18,0x0C,0x00,0x00,0x00,
// 0x2D - -
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xFE,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x2E - .
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x18,0x18,0x00,0x00,0x00,0x00,
// 0x2F - /
0x00,0x00,0x00,0x00,0x01,0x03,0x06,0x0C,0x18,0x30,0x60,0x40,0x00,0x00,0x00,0x00,
// 0x30 - 0
0x00,0x00,0x3C,0x66,0xC3,0xC3,0xDB,0xDB,0xC3,0xC3,0x66,0x3C,0x00,0x00,0x00,0x00,
// 0x31 - 1
0x00,0x00,0x18,0x38,0x78,0x18,0x18,0x18,0x18,0x18,0x18,0x7E,0x00,0x00,0x00,0x00,
// 0x32 - 2
0x00,0x00,0x3C,0x66,0x66,0x06,0x0C,0x18,0x30,0x60,0x66,0x7E,0x00,0x00,0x00,0x00,
// 0x33 - 3
0x00,0x00,0x3C,0x66,0x66,0x06,0x1C,0x06,0x06,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
// 0x34 - 4
0x00,0x00,0x0C,0x1C,0x3C,0x6C,0xCC,0xFE,0x0C,0x0C,0x0C,0x1E,0x00,0x00,0x00,0x00,
// 0x35 - 5
0x00,0x00,0x7E,0x60,0x60,0x60,0x7C,0x06,0x06,0x06,0x66,0x3C,0x00,0x00,0x00,0x00,
// 0x36 - 6
0x00,0x00,0x3C,0x66,0x60,0x60,0x7C,0x66,0x66,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
// 0x37 - 7
0x00,0x00,0x7E,0x66,0x06,0x06,0x0C,0x18,0x18,0x18,0x18,0x18,0x00,0x00,0x00,0x00,
// 0x38 - 8
0x00,0x00,0x3C,0x66,0x66,0x66,0x3C,0x66,0x66,0x66,0x66,0x3C,0x00,0x00,0x00,0x00,
// 0x39 - 9
0x00,0x00,0x3C,0x66,0x66,0x66,0x3E,0x06,0x06,0x06,0x66,0x3C,0x00,0x00,0x00,0x00,
// 0x3A - :
0x00,0x00,0x00,0x00,0x18,0x18,0x00,0x00,0x00,0x18,0x18,0x00,0x00,0x00,0x00,0x00,
// 0x3B - ;
0x00,0x00,0x00,0x00,0x18,0x18,0x00,0x00,0x00,0x18,0x18,0x0C,0x00,0x00,0x00,0x00,
// 0x3C - &lt;
0x00,0x00,0x00,0x06,0x0C,0x18,0x30,0x60,0x30,0x18,0x0C,0x06,0x00,0x00,0x00,0x00,
// 0x3D - =
0x00,0x00,0x00,0x00,0x00,0x7E,0x00,0x00,0x7E,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x3E - &gt;
0x00,0x00,0x00,0x60,0x30,0x18,0x0C,0x06,0x0C,0x18,0x30,0x60,0x00,0x00,0x00,0x00,
// 0x3F - ?
0x00,0x00,0x3C,0x66,0x66,0x06,0x0C,0x18,0x18,0x00,0x18,0x18,0x00,0x00,0x00,0x00,
// 0x40 - @
0x00,0x00,0x00,0x7C,0xC6,0xC6,0xDE,0xDE,0xDE,0xDC,0xC0,0x7C,0x00,0x00,0x00,0x00,
// 0x41 - A
0x00,0x00,0x10,0x38,0x6C,0xC6,0xC6,0xFE,0xC6,0xC6,0xC6,0xC6,0x00,0x00,0x00,0x00,
// 0x42 - B
0x00,0x00,0xFC,0x66,0x66,0x66,0x7C,0x66,0x66,0x66,0x66,0xFC,0x00,0x00,0x00,0x00,
// 0x43 - C
```

```
0x00,0x00,0x3C,0x66,0xC2,0xC0,0xC0,0xC0,0xC0,0xC2,0x66,0x3C,0x00,0x00,0x00,0x00,
// 0x44 - D
0x00,0x00,0xF8,0x6C,0x66,0x66,0x66,0x66,0x66,0x66,0x6C,0xF8,0x00,0x00,0x00,0x00,
// 0x45 - E
0x00,0x00,0xFE,0x66,0x62,0x68,0x78,0x68,0x60,0x62,0x66,0xFE,0x00,0x00,0x00,0x00,
// 0x46 - F
0x00,0x00,0xFE,0x66,0x62,0x68,0x78,0x68,0x60,0x60,0x60,0xF0,0x00,0x00,0x00,0x00,
// 0x47 - G
0x00,0x00,0x3C,0x66,0xC2,0xC0,0xC0,0xDE,0xC6,0xC6,0x66,0x3A,0x00,0x00,0x00,0x00,
// 0x48 - H
0x00,0x00,0xC6,0xC6,0xC6,0xC6,0xFE,0xC6,0xC6,0xC6,0xC6,0xC6,0x00,0x00,0x00,0x00,
// 0x49 - I
0x00,0x00,0x3C,0x18,0x18,0x18,0x18,0x18,0x18,0x18,0x18,0x3C,0x00,0x00,0x00,0x00,
// 0x4A - J
0x00,0x00,0x1E,0x0C,0x0C,0x0C,0x0C,0x0C,0xCC,0xCC,0xCC,0x78,0x00,0x00,0x00,0x00,
// 0x4B - K
0x00,0x00,0xE6,0x66,0x66,0x6C,0x78,0x78,0x6C,0x66,0x66,0xE6,0x00,0x00,0x00,0x00,
// 0x4C - L
0x00,0x00,0xF0,0x60,0x60,0x60,0x60,0x60,0x60,0x62,0x66,0xFE,0x00,0x00,0x00,0x00,
// 0x4D - M
0x00,0x00,0xC6,0xEE,0xFE,0xFE,0xD6,0xC6,0xC6,0xC6,0xC6,0xC6,0x00,0x00,0x00,0x00,
// 0x4E - N
0x00,0x00,0xC6,0xE6,0xF6,0xFE,0xDE,0xCE,0xC6,0xC6,0xC6,0xC6,0x00,0x00,0x00,0x00,
// 0x4F - O
0x00,0x00,0x7C,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0x7C,0x00,0x00,0x00,0x00,
// 0x50 - P
0x00,0x00,0xFC,0x66,0x66,0x66,0x7C,0x60,0x60,0x60,0x60,0xF0,0x00,0x00,0x00,0x00,
// 0x51 - Q
0x00,0x00,0x7C,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0xD6,0xDE,0x7C,0x0C,0x0E,0x00,0x00,
// 0x52 - R
0x00,0x00,0xFC,0x66,0x66,0x66,0x7C,0x6C,0x66,0x66,0x66,0xE6,0x00,0x00,0x00,0x00,
// 0x53 - S
0x00,0x00,0x7C,0xC6,0xC6,0x60,0x38,0x0C,0x06,0xC6,0xC6,0x7C,0x00,0x00,0x00,0x00,
// 0x54 - T
0x00,0x00,0xFF,0xDB,0x99,0x18,0x18,0x18,0x18,0x18,0x18,0x3C,0x00,0x00,0x00,0x00,
// 0x55 - U
0x00,0x00,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0x7C,0x00,0x00,0x00,0x00,
// 0x56 - V
0x00,0x00,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0x6C,0x38,0x10,0x00,0x00,0x00,0x00,
// 0x57 - W
0x00,0x00,0xC6,0xC6,0xC6,0xC6,0xD6,0xD6,0xD6,0xFE,0xEE,0x6C,0x00,0x00,0x00,0x00,
// 0x58 - X
0x00,0x00,0xC6,0xC6,0x6C,0x7C,0x38,0x38,0x7C,0x6C,0xC6,0xC6,0x00,0x00,0x00,0x00,
// 0x59 - Y
0x00,0x00,0xC6,0xC6,0xC6,0x6C,0x38,0x18,0x18,0x18,0x18,0x3C,0x00,0x00,0x00,0x00,
// 0x5A - Z
0x00,0x00,0xFE,0xC6,0x86,0x0C,0x18,0x30,0x60,0xC2,0xC6,0xFE,0x00,0x00,0x00,0x00,
// 0x5B - [
0x00,0x00,0x3C,0x30,0x30,0x30,0x30,0x30,0x30,0x30,0x30,0x3C,0x00,0x00,0x00,0x00,
// 0x5C - backslash
0x00,0x00,0x00,0x80,0xC0,0xE0,0x70,0x38,0x1C,0x0E,0x06,0x02,0x00,0x00,0x00,0x00,
// 0x5D - ]
0x00,0x00,0x3C,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x0C,0x3C,0x00,0x00,0x00,0x00,
// 0x5E - ^
0x10,0x38,0x6C,0xC6,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x5F - _
0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0xFF,0x00,0x00,0x00,
// 0x60 - `
0x30,0x30,0x18,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
// 0x61 - a
0x00,0x00,0x00,0x00,0x00,0x78,0x0C,0x7C,0xCC,0xCC,0xCC,0x76,0x00,0x00,0x00,0x00,
// 0x62 - b
0x00,0x00,0xE0,0x60,0x60,0x78,0x6C,0x66,0x66,0x66,0x66,0x7C,0x00,0x00,0x00,0x00,
// 0x63 - c
0x00,0x00,0x00,0x00,0x00,0x7C,0xC6,0xC0,0xC0,0xC0,0xC6,0x7C,0x00,0x00,0x00,0x00,
```

```
    // 0x64 - d
    0x00,0x00,0x1C,0x0C,0x0C,0x3C,0x6C,0xCC,0xCC,0xCC,0xCC,0x76,0x00,0x00,0x00,0x00,
    // 0x65 - e
    0x00,0x00,0x00,0x00,0x00,0x7C,0xC6,0xFE,0xC0,0xC0,0xC6,0x7C,0x00,0x00,0x00,0x00,
    // 0x66 - f
    0x00,0x00,0x38,0x6C,0x64,0x60,0xF0,0x60,0x60,0x60,0x60,0xF0,0x00,0x00,0x00,0x00,
    // 0x67 - g
    0x00,0x00,0x00,0x00,0x00,0x76,0xCC,0xCC,0xCC,0xCC,0xCC,0x7C,0x0C,0xCC,0x78,0x00,
    // 0x68 - h
    0x00,0x00,0xE0,0x60,0x60,0x6C,0x76,0x66,0x66,0x66,0x66,0xE6,0x00,0x00,0x00,0x00,
    // 0x69 - i
    0x00,0x00,0x18,0x18,0x00,0x38,0x18,0x18,0x18,0x18,0x18,0x3C,0x00,0x00,0x00,0x00,
    // 0x6A - j
    0x00,0x00,0x06,0x06,0x00,0x0E,0x06,0x06,0x06,0x06,0x06,0x06,0x66,0x66,0x3C,0x00,
    // 0x6B - k
    0x00,0x00,0xE0,0x60,0x60,0x66,0x6C,0x78,0x78,0x6C,0x66,0xE6,0x00,0x00,0x00,0x00,
    // 0x6C - l
    0x00,0x00,0x38,0x18,0x18,0x18,0x18,0x18,0x18,0x18,0x18,0x3C,0x00,0x00,0x00,0x00,
    // 0x6D - m
    0x00,0x00,0x00,0x00,0x00,0xEC,0xFE,0xD6,0xD6,0xD6,0xD6,0xC6,0x00,0x00,0x00,0x00,
    // 0x6E - n
    0x00,0x00,0x00,0x00,0x00,0xDC,0x66,0x66,0x66,0x66,0x66,0x66,0x00,0x00,0x00,0x00,
    // 0x6F - o
    0x00,0x00,0x00,0x00,0x00,0x7C,0xC6,0xC6,0xC6,0xC6,0xC6,0x7C,0x00,0x00,0x00,0x00,
    // 0x70 - p
    0x00,0x00,0x00,0x00,0x00,0xDC,0x66,0x66,0x66,0x66,0x66,0x7C,0x60,0x60,0xF0,0x00,
    // 0x71 - q
    0x00,0x00,0x00,0x00,0x00,0x76,0xCC,0xCC,0xCC,0xCC,0xCC,0x7C,0x0C,0x0C,0x1E,0x00,
    // 0x72 - r
    0x00,0x00,0x00,0x00,0x00,0xDC,0x76,0x66,0x60,0x60,0x60,0xF0,0x00,0x00,0x00,0x00,
    // 0x73 - s
    0x00,0x00,0x00,0x00,0x00,0x7C,0xC6,0x60,0x38,0x0C,0xC6,0x7C,0x00,0x00,0x00,0x00,
    // 0x74 - t
    0x00,0x00,0x10,0x30,0x30,0xFC,0x30,0x30,0x30,0x30,0x36,0x1C,0x00,0x00,0x00,0x00,
    // 0x75 - u
    0x00,0x00,0x00,0x00,0x00,0xCC,0xCC,0xCC,0xCC,0xCC,0xCC,0x76,0x00,0x00,0x00,0x00,
    // 0x76 - v
    0x00,0x00,0x00,0x00,0x00,0xC6,0xC6,0xC6,0xC6,0xC6,0x6C,0x38,0x00,0x00,0x00,0x00,
    // 0x77 - w
    0x00,0x00,0x00,0x00,0x00,0xC6,0xC6,0xD6,0xD6,0xD6,0xFE,0x6C,0x00,0x00,0x00,0x00,
    // 0x78 - x
    0x00,0x00,0x00,0x00,0x00,0xC6,0x6C,0x38,0x38,0x38,0x6C,0xC6,0x00,0x00,0x00,0x00,
    // 0x79 - y
    0x00,0x00,0x00,0x00,0x00,0xC6,0xC6,0xC6,0xC6,0xC6,0xC6,0x7E,0x06,0x0C,0xF8,0x00,
    // 0x7A - z
    0x00,0x00,0x00,0x00,0x00,0xFE,0xCC,0x18,0x30,0x60,0xC6,0xFE,0x00,0x00,0x00,0x00,
    // 0x7B - {
    0x00,0x00,0x0E,0x18,0x18,0x18,0x70,0x18,0x18,0x18,0x18,0x0E,0x00,0x00,0x00,0x00,
    // 0x7C - |
    0x00,0x00,0x18,0x18,0x18,0x18,0x00,0x18,0x18,0x18,0x18,0x18,0x00,0x00,0x00,0x00,
    // 0x7D - }
    0x00,0x00,0x70,0x18,0x18,0x18,0x0E,0x18,0x18,0x18,0x18,0x70,0x00,0x00,0x00,0x00,
    // 0x7E - ~
    0x00,0x00,0x76,0xDC,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,0x00,
    // 0x7F - DEL
    0x00,0x00,0x00,0x00,0x10,0x38,0x6C,0xC6,0xC6,0xC6,0xFE,0x00,0x00,0x00,0x00,0x00,
];
```

4.2 Graphical Console

The Console struct tracks cursor position (column, row), rendering area bounds, and colors. It handles character output including newline, carriage return, tab (4-space aligned), and word wrapping at the right

edge. When the cursor moves past the last row, `scroll_up()` copies all pixel rows upward by one glyph height and clears the bottom row.

The `ConsoleFmtWriter` adapter implements `core::fmt::Write`, enabling standard Rust formatting macros (`write!`, `writeln!`) to render directly to the graphical console.

novusos/src/console.rs

```
use crate::color::Color;
use crate::font::{self, GLYPH_HEIGHT, GLYPH_WIDTH};
use crate::framebuffer::Framebuffer;
use core::fmt;

pub struct Console {
    col: usize,
    row: usize,
    cols: usize,
    rows: usize,
    x_offset: usize,
    y_offset: usize,
    area_width: usize,
    area_height: usize,
    fg: Color,
    bg: Color,
}

impl Console {
    pub fn new(
        x_offset: usize,
        y_offset: usize,
        area_width: usize,
        area_height: usize,
        fg: Color,
        bg: Color,
    ) -&gt; Self {
        Self {
            col: 0,
            row: 0,
            cols: area_width / GLYPH_WIDTH,
            rows: area_height / GLYPH_HEIGHT,
            x_offset,
            y_offset,
            area_width,
            area_height,
            fg,
            bg,
        }
    }

    pub fn set_area(&amp;mut self, x: usize, y: usize, w: usize, h: usize) {
        self.x_offset = x;
        self.y_offset = y;
        self.area_width = w;
        self.area_height = h;
        self.cols = w / GLYPH_WIDTH;
        self.rows = h / GLYPH_HEIGHT;
    }

    pub fn clear(&amp;mut self, fb: &amp;mut Framebuffer) {
        fb.fill_rect(self.x_offset, self.y_offset, self.area_width, self.area_height, self.bg);
        self.col = 0;
        self.row = 0;
    }
```

```
pub fn set_colors(&amp;mut self, fg: Color, bg: Color) {
    self.fg = fg;
    self.bg = bg;
}

fn scroll_up(&amp;mut self, fb: &amp;mut Framebuffer) {
    fb.scroll_region_up(
        self.x_offset,
        self.y_offset,
        self.area_width,
        self.area_height,
        GLYPH_HEIGHT,
        self.bg,
    );
    self.row = self.rows - 1;
}

fn newline(&amp;mut self, fb: &amp;mut Framebuffer) {
    self.col = 0;
    self.row += 1;
    if self.row &gt;= self.rows {
        self.scroll_up(fb);
    }
}

pub fn put_char(&amp;mut self, fb: &amp;mut Framebuffer, ch: char) {
    match ch {
        '\n' =&gt; self.newline(fb),
        '\r' =&gt; self.col = 0,
        '\t' =&gt; {
            let spaces = 4 - (self.col % 4);
            for _ in 0..spaces {
                self.put_char(fb, ' ');
            }
        }
        _ =&gt; {
            if self.col &gt;= self.cols {
                self.newline(fb);
            }
            let px = self.x_offset + self.col * GLYPH_WIDTH;
            let py = self.y_offset + self.row * GLYPH_HEIGHT;
            font::render_char(fb, px, py, ch, self.fg, self.bg);
            self.col += 1;
        }
    }
}

pub fn put_str(&amp;mut self, fb: &amp;mut Framebuffer, s: &amp;str) {
    for ch in s.chars() {
        self.put_char(fb, ch);
    }
}

pub fn backspace(&amp;mut self, fb: &amp;mut Framebuffer) {
    if self.col &gt; 0 {
        self.col -= 1;
        let px = self.x_offset + self.col * GLYPH_WIDTH;
        let py = self.y_offset + self.row * GLYPH_HEIGHT;
        font::render_char(fb, px, py, ' ', self.fg, self.bg);
    }
}

pub fn cursor_x(&amp;self) -&gt; usize {
    self.x_offset + self.col * GLYPH_WIDTH
}
```

```
    pub fn cursor_y(&amp;self) -&gt; usize {
        self.y_offset + self.row * GLYPH_HEIGHT
    }

    pub fn col(&amp;self) -&gt; usize {
        self.col
    }

    pub fn fg(&amp;self) -&gt; Color {
        self.fg
    }

    pub fn bg(&amp;self) -&gt; Color {
        self.bg
    }
}

pub struct ConsoleFmtWriter&lt;'a, 'b&gt; {
    pub console: &amp;'a mut Console,
    pub fb: &amp;'b mut Framebuffer,
}

impl&lt;'a, 'b&gt; fmt::Write for ConsoleFmtWriter&lt;'a, 'b&gt; {
    fn write_str(&amp;mut self, s: &amp;str) -&gt; fmt::Result {
        self.console.put_str(self.fb, s);
        Ok(())
    }
}
```

5. Phase 3: Keyboard Input & Interactive Shell

5.1 UEFI SimpleTextInput Polling

Keyboard input uses the UEFI SimpleTextInput protocol accessed via `uefi::system::with_stdin()`. The `poll_key()` function calls `stdin.read_key()` which returns immediately (non-blocking). Keys are classified as printable characters, Enter, Backspace, Escape, or arrow keys.

Because UEFI runs in a polled I/O model (no interrupts for keyboard), the main event loop calls `poll_key()` continuously with a 1ms stall between iterations to prevent CPU spinning.

novusos/src/keyboard.rs

```
use uefi::proto::console::text::{Key, ScanCode};

pub enum KeyEvent {
    Char(char),
    Enter,
    Backspace,
    Escape,
    Up,
    Down,
}

pub fn poll_key() -&gt; Option&lt;KeyEvent&gt; {
    uefi::system::with_stdin(|stdin| {
        let key = stdin.read_key().ok()??;
        match key {
            Key::Printable(ch) =&gt; {
                let c: char = ch.into();
                if c == '\r' || c == '\n' {
                    Some(KeyEvent::Enter)
                } else if c == '\x08' {
                    Some(KeyEvent::Backspace)
                } else {
                    Some(KeyEvent::Char(c))
                }
            }
            Key::Special(scan) =&gt; match scan {
                ScanCode::ESCAPE =&gt; Some(KeyEvent::Escape),
                ScanCode::UP =&gt; Some(KeyEvent::Up),
                ScanCode::DOWN =&gt; Some(KeyEvent::Down),
                _ =&gt; None,
            },
        }
    })
}
```

5.2 Shell Line Editor

The Shell struct maintains a 256-byte line buffer. Printable characters are appended and echoed to the console. Backspace removes the last character (both from buffer and screen). Enter triggers command execution. The prompt `novus>` is displayed after each command completes.

novusos/src/shell.rs

```
use crate::commands::{self, SystemCtx};
use crate::console::{Console, ConsoleFmtWriter};
use crate::framebuffer::Framebuffer;
use crate::keyboard::KeyEvent;
use core::fmt::Write;

const MAX_INPUT: usize = 256;
const PROMPT: &amp;str = "novus&gt; ";

pub struct Shell {
    buf: [u8; MAX_INPUT],
    len: usize,
}

impl Shell {
    pub fn new() -&gt; Self {
        Self {
            buf: [0u8; MAX_INPUT],
            len: 0,
        }
    }

    pub fn draw_prompt(&amp;self, con: &amp;mut Console, fb: &amp;mut Framebuffer) {
        let mut w = ConsoleFmtWriter { console: con, fb };
        let _ = write!(w, "{}", PROMPT);
    }

    pub fn handle_key(
        &amp;mut self,
        key: KeyEvent,
        con: &amp;mut Console,
        fb: &amp;mut Framebuffer,
        ctx: &amp;SystemCtx,
    ) -&gt; bool {
        match key {
            KeyEvent::Char(c) =&gt; {
                if self.len &lt; MAX_INPUT - 1 &amp;&amp; c.is_ascii() {
                    self.buf[self.len] = c as u8;
                    self.len += 1;
                    con.put_char(fb, c);
                }
            }
            KeyEvent::Enter =&gt; {
                {
                    let mut w = ConsoleFmtWriter { console: con, fb };
                    let _ = writeln!(w);
                }
                let cmd = core::str::from_utf8(&amp;self.buf[..self.len]).unwrap_or("");
                let cmd = cmd.trim();
                if !cmd.is_empty() {
                    let should_quit = commands::execute(cmd, con, fb, ctx);
                    if should_quit {
                        return true;
                    }
                }
                self.len = 0;
                self.draw_prompt(con, fb);
            }
            KeyEvent::Backspace =&gt; {
                if self.len &gt; 0 {
                    self.len -= 1;
                    con.backspace(fb);
                }
            }
```

```
                _ =&gt; {}
            }
            false
        }
    }
```

5.3 Command Dispatcher

The command dispatcher splits input by whitespace, matches the first token against known commands, and passes remaining arguments to the handler. Unknown commands produce an error message suggesting 'help'.

6. Phase 4: GUI Desktop

6.1 Desktop: Gradient Background & Status Bar

The desktop background renders a vertical gradient from dark blue (20, 30, 60) at the top to medium blue (35, 50, 100) at the bottom, computed per-scanline using integer linear interpolation. This avoids floating-point operations.

A 24-pixel status bar at the top displays system information: OS version, total RAM, and live uptime. The status bar is redrawn every ~1000 event loop iterations (~1 second) to update the uptime counter.

novusos/src/desktop.rs

```
use crate::color::*;
use crate::font;
use crate::framebuffer::Framebuffer;

const STATUS_BAR_HEIGHT: usize = 24;

pub struct Desktop {
    pub width: usize,
    pub height: usize,
}

impl Desktop {
    pub fn new(width: usize, height: usize) -&gt; Self {
        Self { width, height }
    }

    pub fn draw_background(&amp;self, fb: &amp;mut Framebuffer) {
        let top = DARK_BLUE;
        let bot = MEDIUM_BLUE;

        for y in STATUS_BAR_HEIGHT..self.height {
            let t = y - STATUS_BAR_HEIGHT;
            let range = self.height - STATUS_BAR_HEIGHT;
            let r = top.r as usize + (bot.r as usize - top.r as usize) * t / range;
            let g = top.g as usize + (bot.g as usize - top.g as usize) * t / range;
            let b = top.b as usize + (bot.b as usize - top.b as usize) * t / range;
            let color = Color::new(r as u8, g as u8, b as u8);
            fb.fill_rect(0, y, self.width, 1, color);
        }
    }

    pub fn draw_status_bar(&amp;self, fb: &amp;mut Framebuffer, status_text: &amp;str) {
        fb.fill_rect(0, 0, self.width, STATUS_BAR_HEIGHT, STATUS_BG);
        fb.fill_rect(0, STATUS_BAR_HEIGHT - 1, self.width, 1, BORDER);
        font::render_str(fb, 8, 4, status_text, LIGHT_GRAY, STATUS_BG);
    }

    pub fn status_bar_height(&amp;self) -&gt; usize {
        STATUS_BAR_HEIGHT
    }
}
```

6.2 Terminal Window: Borders, Title Bar, Shadow

The TerminalWindow renders a bordered window with a dark title bar ('Terminal'), a close button indicator, a subtle shadow effect (3px offset black rectangle), and a dark content area. The console is configured to render only within the content area, calculated from the window's position minus borders and padding.

novusos/src/window.rs

```
use crate::color::*;
use crate::font;
use crate::framebuffer::Framebuffer;

const TITLE_HEIGHT: usize = 22;
const BORDER_WIDTH: usize = 1;

pub struct TerminalWindow {
    pub x: usize,
    pub y: usize,
    pub width: usize,
    pub height: usize,
}

impl TerminalWindow {
    pub fn new(x: usize, y: usize, width: usize, height: usize) -&gt; Self {
        Self { x, y, width, height }
    }

    pub fn draw(&amp;self, fb: &amp;mut Framebuffer) {
        fb.fill_rect(
            self.x + 3, self.y + 3,
            self.width, self.height,
            Color::new(0, 0, 0),
        );

        fb.fill_rect(self.x, self.y, self.width, self.height, CONTENT_BG);

        fb.fill_rect(self.x, self.y, self.width, TITLE_HEIGHT, TITLE_BAR);

        font::render_str(
            fb,
            self.x + 8, self.y + 3,
            "Terminal",
            WHITE, TITLE_BAR,
        );

        let close_x = self.x + self.width - 16;
        fb.fill_rect(close_x, self.y + 6, 10, 10, RED);

        fb.fill_rect(self.x, self.y, self.width, BORDER_WIDTH, BORDER);
        fb.fill_rect(self.x, self.y + self.height - BORDER_WIDTH, self.width, BORDER_WIDTH, BORDER);
        fb.fill_rect(self.x, self.y, BORDER_WIDTH, self.height, BORDER);
        fb.fill_rect(self.x + self.width - BORDER_WIDTH, self.y, BORDER_WIDTH, self.height, BORDER);

        fb.fill_rect(self.x, self.y + TITLE_HEIGHT, self.width, BORDER_WIDTH, BORDER);
    }

    pub fn content_x(&amp;self) -&gt; usize {
        self.x + BORDER_WIDTH + 4
    }

    pub fn content_y(&amp;self) -&gt; usize {
        self.y + TITLE_HEIGHT + BORDER_WIDTH + 2
    }
```

```
    pub fn content_width(&amp;self) -&gt; usize {
        self.width - 2 * BORDER_WIDTH - 8
    }

    pub fn content_height(&amp;self) -&gt; usize {
        self.height - TITLE_HEIGHT - 2 * BORDER_WIDTH - 4
    }
}
```

7. Phase 5: Built-in Commands & System Info

7.1 Timer: AArch64 Architectural Timer

Uptime tracking uses the AArch64 architectural timer, which is always available regardless of OS exception level. `CNTFRQ_EL0` provides the timer frequency (typically 62.5 MHz on QEMU), and `CNTVCT_EL0` provides the current count. Dividing the elapsed count by the frequency gives seconds.

novusos/src/timer.rs

```
use core::arch::asm;

fn read_cntfrq() -&gt; u64 {
    let val: u64;
    unsafe { asm!("mrs {}, CNTFRQ_EL0", out(reg) val) };
    val
}

fn read_cntvct() -&gt; u64 {
    let val: u64;
    unsafe { asm!("mrs {}, CNTVCT_EL0", out(reg) val) };
    val
}

pub struct BootTimer {
    freq: u64,
    start: u64,
}

impl BootTimer {
    pub fn new() -&gt; Self {
        Self {
            freq: read_cntfrq(),
            start: read_cntvct(),
        }
    }

    pub fn elapsed_secs(&amp;self) -&gt; u64 {
        let now = read_cntvct();
        if self.freq == 0 {
            return 0;
        }
        (now - self.start) / self.freq
    }

    pub fn format_uptime(&amp;self) -&gt; (u64, u64, u64) {
        let total = self.elapsed_secs();
        let h = total / 3600;
        let m = (total % 3600) / 60;
        let s = total % 60;
        (h, m, s)
    }
}
```

7.2 Memory: UEFI Memory Map Query

The `query_memory()` function calls `uefi::boot::memory_map()` to retrieve the UEFI memory map. It sums total pages and categorizes conventional memory, boot services memory, and loader memory as 'available'. Each page is 4KB, so total_pages * 4096 / 1MB gives megabytes.

novusos/src/memory.rs

```
use uefi::mem::memory_map::{MemoryMap, MemoryType};

pub struct MemInfo {
    pub total_pages: u64,
    pub free_pages: u64,
}

impl MemInfo {
    pub fn total_mb(&amp;self) -&gt; u64 {
        self.total_pages * 4096 / (1024 * 1024)
    }

    pub fn free_mb(&amp;self) -&gt; u64 {
        self.free_pages * 4096 / (1024 * 1024)
    }
}

pub fn query_memory() -&gt; MemInfo {
    let mut total_pages = 0u64;
    let mut free_pages = 0u64;

    if let Ok(map) = uefi::boot::memory_map(MemoryType::LOADER_DATA) {
        for desc in map.entries() {
            total_pages += desc.page_count;
            match desc.ty {
                MemoryType::CONVENTIONAL
                | MemoryType::BOOT_SERVICES_CODE
                | MemoryType::BOOT_SERVICES_DATA
                | MemoryType::LOADER_CODE
                | MemoryType::LOADER_DATA =&gt; {
                    free_pages += desc.page_count;
                }
                _ =&gt; {}
            }
        }
    }

    MemInfo { total_pages, free_pages }
}
```

7.3 CPU Info: MIDR_EL1 Decode

The `cpuinfo` command reads MIDR_EL1 (Main ID Register) and decodes the implementer code (bits 31:24), variant (bits 23:20), architecture (bits 19:16), part number (bits 15:4), and revision (bits 3:0). Known implementers include ARM (0x41), Apple (0x61), Qualcomm (0x51), and NVIDIA (0x4E). Known part numbers include Cortex-A72 (0xD08), Cortex-A76 (0xD0B), and Neoverse-N1 (0xD0C).

7.4 Full Command Reference

Command	Arguments	Description
help	(none)	Display list of available commands
ver	(none)	Show version, architecture, boot mode
echo	<text>	Print text to terminal
clear	(none)	Clear terminal content area
mem	(none)	Show total and available RAM from UEFI memory map
uptime	(none)	Show elapsed time since boot (HH:MM:SS)
cpuinfo	(none)	Decode MIDR_EL1: implementer, part, variant
color	r g b	Change desktop background to RGB color
reboot	(none)	Cold reset via UEFI runtime services
shutdown	(none)	Power off via UEFI runtime services

novusos/src/commands.rs

```
use crate::color::*;
use crate::console::{Console, ConsoleFmtWriter};
use crate::desktop::Desktop;
use crate::framebuffer::Framebuffer;
use crate::memory;
use crate::timer::BootTimer;
use core::fmt::Write;

pub struct SystemCtx&amp;lt;'a&amp;gt; {
    pub timer: &amp;amp;'a BootTimer,
    pub desktop: &amp;amp;'a Desktop,
}

pub fn execute(
    input: &amp;amp;str,
    con: &amp;amp;mut Console,
    fb: &amp;amp;mut Framebuffer,
    ctx: &amp;amp;SystemCtx,
) -&amp;gt; bool {
    let mut parts = input.split_whitespace();
    let cmd = match parts.next() {
        Some(c) =&amp;gt; c,
        None =&amp;gt; return false,
    };
    let args: &amp;amp;str = input[cmd.len()..].trim();

    match cmd {
        "help" =&amp;gt; cmd_help(con, fb),
        "clear" =&amp;gt; cmd_clear(con, fb),
        "echo" =&amp;gt; cmd_echo(args, con, fb),
        "ver" | "version" =&amp;gt; cmd_ver(con, fb),
        "mem" | "memory" =&amp;gt; cmd_mem(con, fb),
        "uptime" =&amp;gt; cmd_uptime(con, fb, ctx),
        "cpuinfo" | "cpu" =&amp;gt; cmd_cpuinfo(con, fb),
        "color" =&amp;gt; cmd_color(args, fb, ctx),
        "reboot" =&amp;gt; cmd_reboot(),
        "shutdown" | "halt" =&amp;gt; cmd_shutdown(),
        _ =&amp;gt; {
```

```
            let mut w = ConsoleFmtWriter { console: con, fb };
            let _ = writeln!(w, "Unknown command: '{}'. Type 'help' for commands.", cmd);
        }
    }
    false
}

fn cmd_help(con: &amp;mut Console, fb: &amp;mut Framebuffer) {
    let mut w = ConsoleFmtWriter { console: con, fb };
    let _ = writeln!(w, "NovusOS Commands:");
    let _ = writeln!(w, "  help          Show this help message");
    let _ = writeln!(w, "  ver           Show version information");
    let _ = writeln!(w, "  echo &lt;text&gt;   Print text");
    let _ = writeln!(w, "  clear         Clear the terminal");
    let _ = writeln!(w, "  mem           Show memory information");
    let _ = writeln!(w, "  uptime        Show system uptime");
    let _ = writeln!(w, "  cpuinfo       Show CPU information");
    let _ = writeln!(w, "  color r g b   Change desktop background");
    let _ = writeln!(w, "  reboot        Restart the system");
    let _ = writeln!(w, "  shutdown      Power off the system");
}

fn cmd_clear(con: &amp;mut Console, fb: &amp;mut Framebuffer) {
    con.clear(fb);
}

fn cmd_echo(args: &amp;str, con: &amp;mut Console, fb: &amp;mut Framebuffer) {
    let mut w = ConsoleFmtWriter { console: con, fb };
    let _ = writeln!(w, "{}", args);
}

fn cmd_ver(con: &amp;mut Console, fb: &amp;mut Framebuffer) {
    let mut w = ConsoleFmtWriter { console: con, fb };
    let _ = writeln!(w, "NovusOS v1.0");
    let _ = writeln!(w, "Architecture: AArch64 (ARM64)");
    let _ = writeln!(w, "Boot: UEFI Application");
    let _ = writeln!(w, "Display: GOP Framebuffer");
}

fn cmd_mem(con: &amp;mut Console, fb: &amp;mut Framebuffer) {
    let info = memory::query_memory();
    let mut w = ConsoleFmtWriter { console: con, fb };
    let _ = writeln!(w, "Memory Information:");
    let _ = writeln!(w, "  Total: {} MB ({} pages)", info.total_mb(), info.total_pages);
    let _ = writeln!(w, "  Available: {} MB ({} pages)", info.free_mb(), info.free_pages);
}

fn cmd_uptime(con: &amp;mut Console, fb: &amp;mut Framebuffer, ctx: &amp;SystemCtx) {
    let (h, m, s) = ctx.timer.format_uptime();
    let mut w = ConsoleFmtWriter { console: con, fb };
    let _ = writeln!(w, "Uptime: {:02}:{:02}:{:02}", h, m, s);
}

fn cmd_cpuinfo(con: &amp;mut Console, fb: &amp;mut Framebuffer) {
    let midr: u64;
    unsafe { core::arch::asm!("mrs {}, MIDR_EL1", out(reg) midr) };

    let implementer = ((midr &gt;&gt; 24) &amp; 0xFF) as u8;
    let variant = ((midr &gt;&gt; 20) &amp; 0xF) as u8;
    let architecture = ((midr &gt;&gt; 16) &amp; 0xF) as u8;
    let part = ((midr &gt;&gt; 4) &amp; 0xFFF) as u16;
    let revision = (midr &amp; 0xF) as u8;

    let impl_name = match implementer {
        0x41 =&gt; "ARM",
```

```
            0x42 =&gt; "Broadcom",
            0x43 =&gt; "Cavium",
            0x44 =&gt; "DEC",
            0x4E =&gt; "NVIDIA",
            0x51 =&gt; "Qualcomm",
            0x56 =&gt; "Marvell",
            0x61 =&gt; "Apple",
            _ =&gt; "Unknown",
        };

        let part_name = match (implementer, part) {
            (0x41, 0xD08) =&gt; "Cortex-A72",
            (0x41, 0xD09) =&gt; "Cortex-A73",
            (0x41, 0xD0A) =&gt; "Cortex-A75",
            (0x41, 0xD0B) =&gt; "Cortex-A76",
            (0x41, 0xD0C) =&gt; "Neoverse-N1",
            (0x41, 0xD05) =&gt; "Cortex-A55",
            (0x41, 0xD03) =&gt; "Cortex-A53",
            _ =&gt; "Unknown",
        };

        let mut w = ConsoleFmtWriter { console: con, fb };
        let _ = writeln!(w, "CPU Information:");
        let _ = writeln!(w, "  Implementer: {} (0x{:02X})", impl_name, implementer);
        let _ = writeln!(w, "  Part: {} (0x{:03X})", part_name, part);
        let _ = writeln!(w, "  Variant: {}, Revision: {}", variant, revision);
        let _ = writeln!(w, "  Architecture: 0x{:X}", architecture);
    }

    fn cmd_color(args: &amp;str, fb: &amp;mut Framebuffer, ctx: &amp;SystemCtx) {
        let parts: alloc::vec::Vec&lt;&amp;str&gt; = args.split_whitespace().collect();
        if parts.len() != 3 {
            return;
        }
        let r: u8 = parts[0].parse().unwrap_or(20);
        let g: u8 = parts[1].parse().unwrap_or(30);
        let b: u8 = parts[2].parse().unwrap_or(60);

        let new_bg = Color::new(r, g, b);
        for y in ctx.desktop.status_bar_height()..ctx.desktop.height {
            fb.fill_rect(0, y, ctx.desktop.width, 1, new_bg);
        }
    }

    fn cmd_reboot() -&gt; ! {
        uefi::runtime::reset(uefi::runtime::ResetType::COLD, uefi::Status::SUCCESS, None);
    }

    fn cmd_shutdown() -&gt; ! {
        uefi::runtime::reset(uefi::runtime::ResetType::SHUTDOWN, uefi::Status::SUCCESS, None);
    }
```

8. Phase 6: ISO Image & VMware Configuration

8.1 Bootable Image Creation

The `tools/mkiso.sh` script creates a bootable disk image using macOS native tools. It creates a FAT32 disk image via `hdiutil`, copies the EFI binary to `/efi/boot/BOOTAA64.EFI`, and adds a `startup.nsh` for automatic boot in the UEFI Shell.

tools/mkiso.sh

```
#!/bin/bash
set -e

EFI_BINARY="target/aarch64-unknown-uefi/release/novusos.efi"
OUTPUT="novusos.img"

echo "Creating NovusOS bootable disk image..."

if ! [ -f "$EFI_BINARY" ]; then
    echo "Error: $EFI_BINARY not found. Run 'make build-novusos' first."
    exit 1
fi

# Create a DMG with FAT32 filesystem using hdiutil
WORK_DIR=$(mktemp -d)

hdiutil create -fs FAT32 -size 64m -volname NOVUSOS \
    -o "$WORK_DIR/novusos" 2&gt;/dev/null

# Mount and add EFI binary
MOUNT_INFO=$(hdiutil attach "$WORK_DIR/novusos.dmg" 2&gt;/dev/null)
MOUNT_POINT=$(echo "$MOUNT_INFO" | grep -o '/Volumes/[^"]*' | head -1)

if [ -n "$MOUNT_POINT" ]; then
    mkdir -p "$MOUNT_POINT/efi/boot"
    cp "$EFI_BINARY" "$MOUNT_POINT/efi/boot/BOOTAA64.EFI"
    printf '\\efi\\boot\\BOOTAA64.EFI\r\n' &gt; "$MOUNT_POINT/startup.nsh"
    sync
    hdiutil detach "$MOUNT_POINT" -quiet 2&gt;/dev/null || true
fi

# Convert to raw format (for QEMU -cdrom and VMware)
hdiutil convert "$WORK_DIR/novusos.dmg" -format UDRW -o "$WORK_DIR/novusos_raw" 2&gt;/dev/null || true

if [ -f "$WORK_DIR/novusos_raw.dmg" ]; then
    cp "$WORK_DIR/novusos_raw.dmg" "$OUTPUT"
else
    cp "$WORK_DIR/novusos.dmg" "$OUTPUT"
fi

rm -rf "$WORK_DIR"

SIZE=$(du -h "$OUTPUT" | cut -f1 | xargs)
echo "Created: $OUTPUT ($SIZE)"
echo ""
echo "Boot options:"
echo "  QEMU:   make run-novusos  (uses VVFAT ESP directory)"
echo "  QEMU:   make run-iso      (uses disk image)"
```

```
echo "  VMware: Attach $OUTPUT as a hard disk or CD/DVD"
```

8.2 QEMU UEFI Boot Configuration

QEMU boots NovusOS using EDK2 UEFI firmware loaded as a pflash drive. The EFI binary is served from a VVFAT directory (fat:rw:esp) that QEMU exposes as a virtual FAT filesystem. The `ramfb` device provides a simple framebuffer, and `qemu-xhci` with `usb-kbd` provides USB keyboard support.

QEMU Launch Command (make run-novusos)

```
qemu-system-aarch64 \
  -machine virt -cpu cortex-a72 -m 256M \
  -drive if=pflash,format=raw,readonly=on,\
         file=/opt/homebrew/share/qemu/edk2-aarch64-code.fd \
  -drive format=raw,file=fat:rw:esp \
  -device ramfb \
  -device qemu-xhci \
  -device usb-kbd
```

8.3 VMware Fusion Setup

VMware Fusion on Apple Silicon natively supports UEFI boot for ARM64 virtual machines. To boot NovusOS:

- **Step 1:** Open VMware Fusion and create a new virtual machine
- **Step 2:** Select 'Other' > 'Other 64-bit Arm' as the guest OS
- **Step 3:** Allocate at least 256 MB RAM
- **Step 4:** Attach novusos.img as a hard disk or create a new disk and copy BOOTAA64.EFI to EFI/BOOT/
- **Step 5:** Boot the VM - UEFI firmware will automatically find and execute BOOTAA64.EFI
- **Step 6:** If the UEFI Shell appears instead, type: FS0:\efi\boot\BOOTAA64.EFI

VMware Fusion uses its own UEFI firmware implementation (based on Tianocore/EDK2), which provides the same GOP and SimpleTextInput protocols that NovusOS depends on.

9. Bugs, Issues & Fixes

This section documents issues encountered during NovusOS development and their resolutions.

1. Workspace Target Conflict

Symptom: Building novusos from the workspace root used the kernel's target (aarch64-unknown-none) instead of aarch64-unknown-uefi.

Root Cause: The root .cargo/config.toml sets the build target to aarch64-unknown-none with a custom linker script. Workspace members inherit this configuration.

Fix: Created novusos/.cargo/config.toml overriding [build] target to aarch64-unknown-uefi. Building with 'cd novusos && cargo build --release' or 'cargo build -p novusos' picks up the per-crate override.

2. UEFI Crate API Version Differences

Symptom: Code using uefi::system::stdin() failed to compile with 'not found in uefi::system'.

Root Cause: The uefi 0.34 crate uses uefi::system::with_stdin() (closure-based API) rather than directly returning a reference. This API prevents the caller from holding the reference across boot service calls.

Fix: Changed keyboard polling to use uefi::system::with_stdin(|stdin| { ... }) pattern, performing the read_key() call inside the closure.

3. Memory Map API Breaking Changes

Symptom: MemoryMapBackingMemory::from_slice() not found; entries() method not available without trait import.

Root Cause: uefi 0.34 changed boot::memory_map() to accept a MemoryType parameter (for backing allocation) and requires importing the MemoryMap trait for the entries() iterator.

Fix: Changed to uefi::boot::memory_map(MemoryType::LOADER_DATA) and added 'use uefi::mem::memory_map::MemoryMap' import.

4. Framebuffer Stride vs Width

Symptom: Drawn pixels appeared at wrong positions; display showed diagonal artifacts.

Root Cause: The pixel offset was calculated using width instead of stride. On some GOP implementations, stride (pixels per scanline including padding) exceeds the visible width.

Fix: Changed offset calculation to (y * stride + x) * 4, using the stride value from GOP mode info rather than the visible width.

5. Borrow Conflict in ConsoleFmtWriter

Symptom: Cannot borrow fb.width while fb is mutably borrowed by ConsoleFmtWriter.

Root Cause: ConsoleFmtWriter borrows both console and fb mutably. Accessing fb.width inside the same scope where the writer holds a mutable reference violates Rust's borrow rules.

Fix: Cache fb.width and fb.height in local variables before creating the ConsoleFmtWriter, using the cached values for formatting.

10. Boot Output & Verification

NovusOS produces both UEFI serial log output and graphical framebuffer output. The serial output is visible in QEMU's terminal and confirms successful initialization:

Expected Display Layout

```
[INFO  novusos] NovusOS v1.0 starting...
[INFO  novusos] Framebuffer: 1024x768, stride=1024
[novusos] Desktop gradient drawn
[novusos] Status bar: NovusOS v1.0 | RAM: 256 MB | 1024x768
[novusos] Terminal window drawn
[novusos] Shell ready

Graphical Output (framebuffer):
  +-----------------------------------------+
  | NovusOS v1.0 | RAM: 256 MB | Uptime: 00: |
  +=========================================+
  |                                         |
  |   +--- Terminal ----------------[x]+    |
  |   | NovusOS v1.0 -- AArch64 UEFI  |    |
  |   | Framebuffer: 1024x768         |    |
  |   | Type 'help' for commands.     |    |
  |   |                               |    |
  |   | novus&gt; help                   |    |
  |   | NovusOS Commands:             |    |
  |   |   help     Show this help     |    |
  |   |   ver      Show version info  |    |
  |   |   mem      Show memory info   |    |
  |   |   uptime   Show system uptime |    |
  |   |   cpuinfo  Show CPU info      |    |
  |   |   ...                         |    |
  |   | novus&gt; _                      |    |
  |   +-------------------------------+    |
  |         (gradient background)           |
  +-----------------------------------------+
```

Verification Checklist

Phase	Status	Evidence
Phase 1: UEFI Boot + GOP	PASS	Framebuffer initialized, pixels drawn
Phase 2: Font + Console	PASS	Text rendered on framebuffer via fmt::Write
Phase 3: Keyboard + Shell	PASS	Interactive prompt, help command works
Phase 4: GUI Desktop	PASS	Gradient bg, status bar, terminal window
Phase 5: Commands	PASS	mem, uptime, cpuinfo, reboot all functional
Phase 6: ISO + VMware	PASS	Bootable image created, QEMU boot verified

11. Appendix: Makefile & Build Configuration

The complete Makefile includes targets for both the bare-metal kernel and NovusOS. NovusOS-specific targets: `build-novusos`, `run-novusos`, `iso`, and `run-iso`.

Makefile

```
KERNEL_ELF = target/aarch64-unknown-none/release/kernel
KERNEL_BIN = target/kernel.bin
DISK_IMG = disk.img

QEMU = qemu-system-aarch64
QEMU_ARGS = -machine virt,gic-version=3 -cpu cortex-a72 -m 256M -nographic -serial mon:stdio
QEMU_DEVICES = -drive file=$(DISK_IMG),format=raw,if=none,id=disk0 \
■-device virtio-blk-device,drive=disk0 \
■-netdev user,id=net0 \
■-device virtio-net-device,netdev=net0

NOVUSOS_EFI = target/aarch64-unknown-uefi/release/novusos.efi
UEFI_FW = /opt/homebrew/share/qemu/edk2-aarch64-code.fd
NOVUSOS_IMG = novusos.img

.PHONY: build run run-bare debug clean clippy disk build-novusos run-novusos iso run-iso

build:
■cargo build --release

$(KERNEL_BIN): build
■rust-objcopy --strip-all -O binary $(KERNEL_ELF) $(KERNEL_BIN)

disk:
■./tools/mkimage.sh $(DISK_IMG)

run: $(KERNEL_BIN) $(DISK_IMG)
■$(QEMU) $(QEMU_ARGS) $(QEMU_DEVICES) -kernel $(KERNEL_BIN)

run-bare: $(KERNEL_BIN)
■$(QEMU) $(QEMU_ARGS) -kernel $(KERNEL_BIN)

debug: $(KERNEL_BIN) $(DISK_IMG)
■$(QEMU) $(QEMU_ARGS) $(QEMU_DEVICES) -kernel $(KERNEL_BIN) -S -s

clean:
■cargo clean
■rm -f $(KERNEL_BIN)

clippy:
■cargo clippy --release

build-novusos:
■cd novusos &amp;amp;&amp;amp; cargo build --release
■@mkdir -p esp/efi/boot
■cp $(NOVUSOS_EFI) esp/efi/boot/BOOTAA64.EFI

run-novusos: build-novusos
■$(QEMU) -machine virt -cpu cortex-a72 -m 256M \
■■-drive if=pflash,format=raw,readonly=on,file=$(UEFI_FW) \
■■-drive format=raw,file=fat:rw:esp \
■■-device ramfb \
■■-device qemu-xhci \
```

```
■■-device usb-kbd

iso: build-novusos
■./tools/mkiso.sh

run-iso: iso
■$(QEMU) -machine virt -cpu cortex-a72 -m 256M \
■■-drive if=pflash,format=raw,readonly=on,file=$(UEFI_FW) \
■■-drive file=$(NOVUSOS_IMG),format=raw,if=virtio \
■■-device ramfb \
■■-device qemu-xhci \
■■-device usb-kbd
```

UEFI Entry Point (main.rs)

The complete main.rs orchestrates all phases: GOP initialization, desktop rendering, terminal window setup, console configuration, shell creation, and the event loop with keyboard polling and status bar updates.

novusos/src/main.rs

```
#![no_main]
#![no_std]

extern crate alloc;

mod color;
mod commands;
mod console;
mod desktop;
mod font;
mod framebuffer;
mod keyboard;
mod memory;
mod shell;
mod timer;
mod window;

use alloc::format;
use core::fmt::Write;
use uefi::prelude::*;
use uefi::proto::console::gop::GraphicsOutput;
use framebuffer::Framebuffer;
use commands::SystemCtx;
use console::{Console, ConsoleFmtWriter};
use desktop::Desktop;
use shell::Shell;
use timer::BootTimer;
use window::TerminalWindow;
use color::*;

#[entry]
fn main() -&gt; Status {
    uefi::helpers::init().unwrap();

    let fb = init_gop();
    let mut fb = match fb {
        Some(fb) =&gt; fb,
        None =&gt; {
            log::error!("Failed to initialize GOP framebuffer");
            boot::stall(5_000_000);
            return Status::ABORTED;
        }
```

```
};

let scr_w = fb.width;
let scr_h = fb.height;

let boot_timer = BootTimer::new();
let mem_info = memory::query_memory();

let desk = Desktop::new(scr_w, scr_h);
desk.draw_background(&amp;mut fb);

let status = format!(
    "NovusOS v1.0  |  RAM: {} MB  |  {}x{}",
    mem_info.total_mb(), scr_w, scr_h
);
desk.draw_status_bar(&amp;mut fb, &amp;status);

let margin = 40;
let bar_h = desk.status_bar_height();
let win = TerminalWindow::new(
    margin,
    bar_h + margin / 2,
    scr_w - 2 * margin,
    scr_h - bar_h - margin,
);
win.draw(&amp;mut fb);

let mut con = Console::new(
    win.content_x(),
    win.content_y(),
    win.content_width(),
    win.content_height(),
    WHITE,
    CONTENT_BG,
);

{
    let mut wr = ConsoleFmtWriter { console: &amp;mut con, fb: &amp;mut fb };
    let _ = writeln!(wr, "NovusOS v1.0 -- AArch64 UEFI Graphical OS");
    let _ = writeln!(wr, "Framebuffer: {}x{}  RAM: {} MB", scr_w, scr_h, mem_info.total_mb());
    let _ = writeln!(wr, "Type 'help' for available commands.");
    let _ = writeln!(wr);
}

let mut shell = Shell::new();
shell.draw_prompt(&amp;mut con, &amp;mut fb);

let sys_ctx = SystemCtx {
    timer: &amp;boot_timer,
    desktop: &amp;desk,
};

let mut tick = 0u64;
loop {
    if let Some(key) = keyboard::poll_key() {
        let quit = shell.handle_key(key, &amp;mut con, &amp;mut fb, &amp;sys_ctx);
        if quit {
            break;
        }
    }

    tick += 1;
    if tick % 1000 == 0 {
        let (h, m, s) = boot_timer.format_uptime();
        let status = format!(
```

```
                "NovusOS v1.0  |  RAM: {} MB  |  Uptime: {:02}:{:02}:{:02}",
                mem_info.total_mb(), h, m, s
            );
            desk.draw_status_bar(&amp;mut fb, &amp;status);
        }

        boot::stall(1_000);
    }

    Status::SUCCESS
}

fn init_gop() -&gt; Option&lt;Framebuffer&gt; {
    let handle = boot::get_handle_for_protocol::&lt;GraphicsOutput&gt;().ok()?;
    let mut gop = boot::open_protocol_exclusive::&lt;GraphicsOutput&gt;(handle).ok()?;

    let mut best_mode = None;
    let mut best_res = (0usize, 0usize);

    for mode in gop.modes() {
        let info = mode.info();
        let (w, h) = info.resolution();
        if w == 1024 &amp;&amp; h == 768 {
            best_mode = Some(mode);
            break;
        }
        if w * h &gt; best_res.0 * best_res.1 {
            best_mode = Some(mode);
            best_res = (w, h);
        }
    }

    if let Some(mode) = best_mode {
        let _ = gop.set_mode(&amp;mode);
    }

    let mode_info = gop.current_mode_info();
    let (width, height) = mode_info.resolution();
    let stride = mode_info.stride();
    let pixel_format = mode_info.pixel_format();

    let mut frame_buffer = gop.frame_buffer();
    let base = frame_buffer.as_mut_ptr();

    Some(Framebuffer::new(base, width, height, stride, pixel_format))
}
```

Workspace Configuration

Cargo.toml (workspace root)

```
[workspace]
members = ["kernel", "novusos"]
exclude = ["userspace/init"]
resolver = "2"

[profile.dev]
panic = "abort"

[profile.release]
panic = "abort"
lto = true
opt-level = 2
```

rust-toolchain.toml

```
[toolchain]
channel = "nightly"
targets = ["aarch64-unknown-none", "aarch64-unknown-uefi"]
components = ["rust-src", "llvm-tools-preview", "rustfmt", "clippy"]
```